Capacitación en IA de Kinvecta: guía maestra

Usa bien y con seguridad la IA generativa en el trabajo: 8 sesiones y 2 inmersiones técnicas

VIDEOS	10 (8 sesiones principales + 2 inmersiones técnicas), unas 2 h 01 min en total
LISTA DE REPRODUCCIÓN	Capacitación en IA de Kinvecta (en español): IA generativa en el trabajo, bien y con seguridad (YouTube; enlaces por agregar al publicar)
PÚBLICO	Todos (Sesiones 1–4), quienes automatizan con IA (5–8), quienes construyen y revisan (Inmersiones técnicas A–B)
EN CLASE	Sesiones de 90 minutos; inmersiones técnicas de 120 minutos con laboratorios prácticos

Sobre la Capacitación en IA de Kinvecta

La Capacitación en IA de Kinvecta es un curso gratuito de 10 videos para usar bien y con seguridad la IA generativa en el trabajo: una serie principal de 8 sesiones más dos inmersiones técnicas. Esta es la edición en español de México.

- **Edición en español:** los 10 videos están listos, unas 2 h 01 min en total. Cada sesión indica la duración de su video y sus capítulos; los enlaces de YouTube se agregan al publicar. Lista de reproducción en español: "Capacitación en IA de Kinvecta (en español): IA generativa en el trabajo, bien y con seguridad".
- **Edición en inglés:** Kinvecta AI Training: Using Generative AI Well and Safely at Work (lista de reproducción de YouTube, en inglés), unos 1 h 42 min en total.
- **Para quién es:** cualquier persona que usa IA en el trabajo (Sesiones 1–4), quienes automatizan con IA (Sesiones 5–8) y quienes construyen y revisan sistemas (Inmersiones técnicas A y B). Cada sesión también es un módulo de clase de 90 minutos para quienes facilitan.
- **Etapas:** la serie sigue cinco etapas de uso de la IA: 1 uso diario seguro, 2 la API en un contenedor, 3 flujos de trabajo fijos, 4 llamadas a herramientas, 5 agentes y skills. Todos empiezan en la Etapa 1.
- **Cómo usar esta guía:** empieza con la tabla de abajo y luego abre cualquier sesión para ver sus capítulos y puntos clave. Cada sesión termina con una regla de oro; las diez están en la tabla.

Kinvecta: capacitación, eventos y comunidad de IA · kinvecta.ai

La serie de un vistazo

Diez videos, en orden desde la Sesión 1. Las Sesiones 1–4 son para todos, las Sesiones 5–8 para quienes automatizan trabajo con IA, y las inmersiones técnicas para quienes construyen.

#	Sesión	Video	Para quién	Regla de oro
1	¿Qué es un LLM? Modelos, arneses, chatbots, flujos de trabajo y agentes	12:37	Todos	El modelo piensa. Las skills le enseñan cómo. Las herramientas le permiten actuar. Y el arnés decide cuáles le da.
2	Resiliencia en IA: cambia de modelo o de proveedor sin reconstruir	9:48	Todos	Renta el modelo. Quédate con las conexiones. Quédate con el know-how. Y quédate con las pruebas.
3	Prompts seguros y verificación	7:52	Todos	Herramienta aprobada para cada dato. Dale contexto como a un colega. Revisa cada afirmación. Y una persona firma.
4	Tokens, contexto y cómo encuentra la IA tus datos	8:17	Todos	Manda menos. Guarda en caché lo que no cambia. Usa el buscador correcto.
5	APIs, endpoints y contenedores	7:43	Quienes automatizan	Llámalo desde tu código. Deja el endpoint en la configuración. Séllalo en un contenedor.
6	Cómo construir flujos de trabajo	8:20	Quienes automatizan	Fija la ruta. Revisa cada resultado. Una persona aprueba antes de entregar.
7	Herramientas, MCP e inyección de prompts	9:12	Quienes automatizan	Las herramientas de lectura corren solas, desde fuentes aprobadas. Las de acción esperan a una persona. Todo lo que regresa es dato.
8	Agentes, skills y cómo dirigir el programa	8:17	Quienes automatizan y quienes dirigen	Escríbelo una vez, como skill. Controla lo que sale. Mide lo que cambia.
A	Inmersión técnica A: construye un servidor MCP desde cero	25:21	Quienes construyen	Pocas herramientas, esquemas estrictos. Una persona confirma cada acción. Valida tu propio token. Nunca lo reenvíes.
B	Inmersión técnica B: DevSecOps, contenedores, escaneo y el pipeline de CI/CD	23:39	Quienes construyen y revisan	Fija todas las versiones. Escanea antes de publicar. Firma y publica el mismo digest.

Los títulos de YouTube en español siguen el patrón "<tema> | Capacitación en IA #N" (por ejemplo, "Qué es un LLM: modelos, chatbots, flujos y agentes | Capacitación en IA #1"). Duración total de la edición en español: 2 h 01 min en 10 videos (sesiones principales: 1 h 12 min; inmersiones técnicas: 49 min).

Serie principal: Sesiones 1–8

Cada sesión tiene un video corto, sus puntos clave, la lista de temas y seis preguntas de repaso con respuestas modelo.

Sesión 1 · ¿Qué es un LLM? Modelos, arneses, chatbots, flujos de trabajo y agentes

Video: 12:37 (enlace por agregar al publicar) · Para todos · YouTube: "Qué es un LLM: modelos, chatbots, flujos y agentes | Capacitación en IA #1"

Puntos clave

- Un LLM (modelo de lenguaje grande) es un motor entrenado que predice y escribe texto, un token (un pedazo de texto) a la vez. Sus pesos se congelan después del entrenamiento, así que no aprende de tus chats; la "memoria" es la app guardando notas y volviéndolas a meter.
- El arnés (en inglés, harness) es el software que rodea al modelo. Decide qué puede ver y hacer el modelo, así que el mismo modelo puede ser seguro en una app y riesgoso en otra.
- Hay seis tipos de arnés: asistentes de chat, IA integrada en herramientas, plataformas de la organización, herramientas de código y de agentes, agentes de código abierto y tu propio software por la API.
- Chatbot, flujo de trabajo, agente: la persona elige cada paso, quien diseña fija los pasos por adelantado, o la IA elige los pasos en tiempo de ejecución. Elige un flujo de trabajo cuando los pasos se conocen.

Capítulos: 0:00 Bienvenida: el modelo y el arnés · 0:27 Lo que vas a poder hacer · 0:54 Por qué empezar por el vocabulario · 1:31 Qué es un LLM: predecir el siguiente token · 2:26 Cómo se hace un modelo · 3:03 Seis creencias que hay que corregir · 3:49 Una solicitud pasa por varios modelos · 4:30 La regla de oro · 4:48 Las partes de un sistema de IA · 5:39 El mismo modelo en tres arneses · 6:24 Los seis tipos de arnés · 7:18 Cómo comparar arneses: alcance y autonomía · 7:54 Tres puertas de entrada a la API · 8:38 Chatbot, flujo de trabajo y agente · 9:35 Hasta dónde puedes llegar · 10:17 Ejercicio: clasifica tareas e inventaría un arnés · 11:10 Pon a prueba lo aprendido · 11:50 Lo que sigue: Resiliencia en IA (Sesión 2)

Pon a prueba lo aprendido

1. En una oración, ¿qué es un modelo de lenguaje grande?
   - Un motor entrenado que predice y produce texto, un token a la vez.
2. ¿Por qué el mismo modelo puede ser seguro en una app y riesgoso en otra?
   - El arnés decide qué puede ver y hacer. Seguridad aprueba el arnés, no solo el modelo.
3. ¿El modelo aprende de tus chats? Entonces, ¿qué es la "memoria"?
   - No. Los pesos están congelados después del entrenamiento. La memoria es el arnés guardando notas y volviéndolas a meter.
4. Nombra tres tipos de arnés y un ejemplo de cada uno.
   - Tres de los seis: asistentes de chat, IA integrada en herramientas, plataformas o gateways de la organización, herramientas de código y de agentes, agentes de código abierto, tu propio software por la API.

5. ¿Quién elige cada paso en un chatbot, un flujo de trabajo y un agente?
   - La persona; quien lo diseña, por adelantado; la IA, en tiempo de ejecución.
6. ¿Cuándo elegirías un flujo de trabajo en vez de un agente?
   - Cuando los pasos se conocen de antemano. Es más predecible y más fácil de autorizar y probar.

Sesión 2 · Resiliencia en IA: cambia de modelo o de proveedor sin reconstruir

Video: 9:48 (enlace por agregar al publicar) · Para todos · YouTube: "Resiliencia en IA: cambia de modelo sin reconstruir | Capacitación en IA #2"

Puntos clave

- Ser agnóstico significa que puedes cambiar el modelo, o todo el proveedor, sin reconstruir tu trabajo. Hay cuatro tipos de cambio: versión del modelo, proveedor, endpoint (la dirección a la que tu código manda las solicitudes) y arnés.
- Siete fundamentos facilitan el cambio: herramientas MCP aprobadas, contenedores, escaneo, skills, paquetes de código, una capa de API multiproveedor y evals guardadas (evaluaciones guardadas).
- Deja el nombre del modelo y el endpoint en la configuración, con la versión fijada, para que cambiar sea un cambio de una línea.
- Las evals guardadas son tus pruebas. Detectan la deriva: cuando un modelo nuevo responde distinto y cuando el negocio o los datos cambian bajo un prompt que sigue igual.

Capítulos: 0:00 Bienvenida: ¿qué es tuyo y sobrevive a un cambio? · 0:30 Lo que vas a poder hacer · 0:56 Por qué importa la resiliencia · 1:25 Rentar o ser dueño · 2:04 Cuatro niveles de cambio · 2:44 Cuatro creencias que hay que corregir · 3:21 Siete fundamentos, un ciclo · 4:16 Las siete cosas que son tuyas · 5:02 Cambia de modelo en una sola línea · 5:44 Una skill, escrita una vez, en cada arnés · 6:21 Las evals te dicen si un cambio funcionó · 7:01 La regla de oro · 7:20 Manual de cambio: prueba antes de cambiar · 7:54 Ejercicio: auditoría de dependencia · 8:34 Pon a prueba lo aprendido · 9:13 Lo que sigue: Prompts seguros y verificación (Sesión 3)

Pon a prueba lo aprendido

1. En una oración, ¿qué hace agnóstica a una organización?
   - Puede cambiar el modelo, o todo el proveedor, sin reconstruir su trabajo.
2. Nombra cuatro de los siete fundamentos.
   - Cuatro de estos: herramientas MCP aprobadas, contenedores, escaneo, skills, paquetes de código, una capa de API multiproveedor, evals guardadas.
3. ¿Por qué las evals guardadas importan tanto al cambiar?
   - Son la única evidencia de que el modelo nuevo funciona tan bien como el anterior. Sin ellas, un cambio es una apuesta.
4. ¿Qué diferencia hay entre la deriva del modelo y la deriva del caso de uso?
   - Deriva del modelo: una versión nueva del modelo responde distinto. Deriva del caso de uso: el negocio, los datos o los usuarios cambian mientras el prompt sigue igual.
5. ¿Dónde deben vivir el nombre del modelo y el endpoint?
   - En la configuración, con la versión fijada, no escritos en el código.

6. ¿Por qué probar una skill en cada arnés si el formato es el mismo?
    - Los scripts, las rutas de archivos y los nombres de herramientas pueden cambiar entre arneses, así que una skill puede fallar aunque el formato sea el mismo.

Sesión 3 · Prompts seguros y verificación

Video: 7:52 (enlace por agregar al publicar) · Para todos · YouTube: "Prompts seguros: usa la IA en el trabajo sin quemarte | Capacitación en IA #3"

Puntos clave

- La regla clave: los datos solo van a una herramienta aprobada para su nivel de sensibilidad. Si pegas algo que no debías, detente, repórtalo de inmediato y registra qué compartiste.
- Escribe el prompt (la instrucción que le das al modelo) como si pusieras al tanto a un colega: rol, tarea, público y formato, más el material fuente. Pegar la fuente hace que el modelo trabaje con tu material y no con su memoria.
- Que suene bien no significa que sea correcto. Revisa cada afirmación: fuentes, cifras, nombres y fechas, citas, lo que falta, y tono y sesgo.
- La IA redacta; una persona firma. Un prompt que funciona se vuelve una instrucción guardada, revisada y probada en la biblioteca del equipo.

Capítulos: 0:00 Bienvenida: hábitos para usar la IA con seguridad · 0:29 Lo que vas a poder hacer · 0:50 Por qué importa: hábitos, no tecnología · 1:23 La regla clave: herramienta aprobada para cada dato · 2:02 Niveles de datos y herramientas · 2:33 Si pegaste algo que no debías · 2:59 La IA redacta, una persona firma · 3:30 Prompts con contexto, como para un colega · 4:16 Verificación: revisa cada afirmación · 4:55 Lista de verificación de seis puntos · 5:19 Guarda lo que funciona · 5:58 La regla de oro · 6:17 Ejercicio: dale contexto, revísalo, guárdalo · 6:52 Pon a prueba lo aprendido · 7:29 Lo que sigue: Tokens, contexto y cómo encuentra la IA tus datos (Sesión 4)

Pon a prueba lo aprendido

1. ¿Cuál es la regla clave?
    - Los datos solo van a una herramienta aprobada para su nivel de sensibilidad.
2. ¿Qué haces justo después de pegar algo que no debías?
    - Detenerte, reportarlo de inmediato a tu contacto de seguridad o privacidad, registrar qué compartiste y luego seguir el proceso de incidentes.
3. ¿Qué partes tiene un prompt con contexto?
    - Rol, tarea, público y formato, más el material fuente.
4. Menciona cuatro cosas que revisar antes de usar un borrador.
    - Cuatro de estas: fuentes, cifras, nombres y fechas, citas, lo que falta, tono y sesgo.
5. ¿Quién es responsable del trabajo hecho con ayuda de IA?
    - Una persona responsable. La IA redacta; una persona firma.
6. ¿Qué necesita un prompt para entrar a la biblioteca del equipo?
    - Un encabezado (nombre, propósito, responsable, entradas, nivel de datos), la revisión de otra persona y pasar los casos de prueba.

Sesión 4 · Tokens, contexto y cómo encuentra la IA tus datos

Video: 8:17 (enlace por agregar al publicar) · Para todos · YouTube: "Tokens, contexto y cómo encuentra la IA tus datos | Capacitación en IA #4"

Puntos clave

- Un token son más o menos cuatro caracteres, unas tres cuartas partes de una palabra en inglés. Los códigos y las cifras usan más, y cada proveedor cuenta distinto.
- La ventana de contexto es un solo presupuesto fijo que comparte todo: instrucciones, herramientas, historial, documentos, la pregunta y la respuesta. Lo que mandas cuesta más que lo que recibes.
- Los chats largos salen más caros en cada turno porque se reenvía toda la conversación. El caché abarata el inicio que no cambia; empieza un chat nuevo cuando cambie el tema.
- El modelo solo ve lo que le regresan los buscadores. Usa el buscador correcto para cada dato, y recuerda que un índice de búsqueda es una copia de tus datos que necesita las mismas reglas de acceso.

Capítulos: 0:00 Bienvenida: ¿qué puede ver el modelo? · 0:34 Lo que vas a poder hacer · 0:57 Tres quejas, una sola causa · 1:24 Qué es un token · 2:01 Tipos de tokens · 2:37 La ventana de contexto: un solo presupuesto · 3:20 Por qué los chats largos cuestan más · 3:52 Seis razones por las que importan los tokens · 4:26 Cómo encuentra la IA tus datos · 5:18 Seis formas en que la IA obtiene datos · 5:59 Elige el buscador según los datos · 6:27 La regla de oro · 6:40 Ejercicio: cuéntalo y encuéntralo · 7:20 Pon a prueba lo aprendido · 7:53 Lo que sigue: APIs, endpoints y contenedores (Sesión 5)

Pon a prueba lo aprendido

1. ¿Más o menos cuánto texto es un token?
   - Unos cuatro caracteres, o tres cuartas partes de una palabra en inglés. Los códigos y las cifras usan más.
2. ¿Por qué los chats largos cuestan más en cada turno?
   - Cada turno reenvía el prompt de sistema y todo el historial.
3. ¿Qué comparte la ventana de contexto?
   - Todo lo que se manda y la respuesta: instrucciones, herramientas, historial, documentos, la pregunta y la salida.
4. ¿Qué abarata el caché?
   - Un prefijo estable que se repite: instrucciones, definiciones de herramientas, documentos de referencia.
5. Menciona tres formas en que la IA obtiene datos.
   - Tres de estas: todo en el contexto, búsqueda semántica, búsqueda por palabras clave, búsqueda agéntica, consultas estructuradas, recuperación multimodal.
6. ¿Por qué un índice de búsqueda es un tema de seguridad?
   - Es una copia de los datos, muchas veces guardada en otro lado, así que necesita las mismas reglas de acceso y aprobaciones que la fuente.

Sesión 5 · APIs, endpoints y contenedores

Video: 7:43 (enlace por agregar al publicar) · Para quienes automatizan con IA · YouTube: "APIs, endpoints y contenedores: la IA desde tu código | Capacitación en IA #5"

Puntos clave

- Cuando tu código llama al modelo por la API (la puerta por la que tu software llama al modelo), tu software se vuelve el arnés. Los seis trabajos que hacía la app de chat ahora le tocan a tu equipo: instrucciones, manejo de datos, registro, llaves y red, revisión y límites de costo.
- Hay tres puertas de entrada al mismo modelo: la API del propio proveedor, la versión alojada en un proveedor de nube y la plataforma aprobada de tu organización. Elige primero la puerta, por sus términos y aprobaciones, y luego el modelo.
- Cuatro hábitos hacen que cambiar sea cosa de una línea: el endpoint en la configuración, la versión del modelo fijada, una sola forma de solicitud y una lista de permitidos con ese único endpoint.
- Todo arnés corre sellado en un contenedor con una sola salida. Cada ejecución deja registro: secretos en tiempo de ejecución, un modelo fijado, un registro de auditoría y límites de uso y de costo.

Capítulos: 0:00 Bienvenida: ¿de quién son los controles? · 0:32 Lo que vas a poder hacer · 0:54 Tres problemas que empiezan cuando el código llama al modelo · 1:27 Con la API, tu código se vuelve el arnés · 1:59 Seis trabajos que ahora son de tu código · 2:30 Tres puertas de entrada al mismo modelo · 3:07 Cuatro hábitos: el endpoint en la configuración · 3:30 Sellado en un contenedor, con una sola salida · 4:15 Los cuatro controles de ejecución · 4:50 Lo que puedes hacer hoy · 5:16 La regla de oro · 5:34 Ejercicio: léelo y enciérralo · 6:15 Pon a prueba lo aprendido · 6:56 Lo que sigue: cómo construir flujos de trabajo (Sesión 6) · 7:15 Para quien facilita: el sandbox de la Etapa 2

Pon a prueba lo aprendido

1. Cuando tu código llama al modelo en lugar de la app de chat, ¿de qué tiene que hacerse cargo tu equipo?
   - De los seis trabajos que hacía la app de chat: instrucciones (prompts guardados), manejo de datos, registro, llaves y red, revisión humana y límites de costo.
2. Nombra los tres tipos de endpoint.
   - La API del propio proveedor, la versión alojada en un proveedor de nube y una plataforma aprobada o gateway.
3. ¿Qué eliges primero, la puerta o el modelo, y por qué?
   - La puerta. El modelo es el mismo detrás de cada puerta; lo que cambia es el contrato: términos, retención de datos y aprobación.
4. ¿Por qué dejar el endpoint en la configuración?
   - Para que cambiar sea un cambio de una línea y la lista de permitidos de la red coincida con él.
5. ¿A qué puede llegar un contenedor?
   - Solo al endpoint aprobado del modelo y a los servidores de herramientas aprobados. Todo lo demás, incluidos los archivos de la propia computadora, se bloquea.

6. Nombra los cuatro controles de ejecución.
    - Secretos inyectados en tiempo de ejecución desde un gestor de secretos, versión del modelo fijada, registro de auditoría de cada ejecución y límites de uso y de costo.

Sesión 6 · Cómo construir flujos de trabajo

Video: 8:20 (enlace por agregar al publicar) · Para quienes automatizan con IA · YouTube: "Flujos de trabajo con IA: pasos fijos y aprobación humana | Capacitación en IA #6"

Puntos clave

- En un flujo de trabajo, quien lo diseña fija la ruta antes de que corra. En un chatbot la persona elige cada paso; en un agente, la IA.
- Chat más una API alcanza para la Etapa 3: automatización que funciona, en un contenedor. No necesitas agentes para empezar.
- Un flujo de trabajo tiene seis pasos fijos, con una revisión y una persona: disparador, traer las entradas, prompt guardado, validar, revisión humana y entregar. Cada paso escribe en el registro de auditoría.
- Un resultado malo se marca, nunca se pasa. Una versión nueva del prompt es una liberación nueva: se prueba con casos de referencia y un caso difícil antes de salir.

Capítulos: 0:00 Bienvenida: tres lunes, una ruta fija · 0:38 Lo que vas a poder hacer · 1:03 Por qué importa: tres quejas comunes · 1:34 ¿Quién elige cada paso? Chatbot, flujo y agente · 2:18 Hasta dónde puedes subir con lo que tienes hoy · 2:58 Elige tu primer flujo de trabajo · 3:26 Anatomía de un flujo de trabajo · 4:14 Validación: revisa cada resultado · 4:49 Ejecución segura: sellado, registrado y con límites · 5:22 Antes de liberar: pruébalo y dale versión · 5:55 Mídelo: cuatro números · 6:11 La regla de oro · 6:27 Ejercicio: diséñalo y luego rómpelo · 7:12 Pon a prueba lo aprendido · 7:55 Lo que sigue: Herramientas, MCP e inyección de prompts (Sesión 7)

Pon a prueba lo aprendido

1. ¿Quién decide los pasos en un flujo de trabajo?
    - Quien lo diseña, por adelantado. El script sigue la misma ruta en cada ejecución.
2. ¿Qué puede construir un equipo solo con chat y una API?
    - Flujos con scripts o low-code que llaman al modelo por la API, con servidores MCP como pasos fijos. Eso es la Etapa 3.
3. Nombra tres señales de un buen primer flujo de trabajo.
    - Tres de estas: se repite cada semana, tiene la misma forma cada vez, hay un resultado de referencia, ya se revisa, el nivel de sensibilidad está aprobado, el tiempo se puede medir.
4. ¿Qué pasa cuando falla una revisión de validación?
    - La ejecución se marca y se manda a una persona. No se entrega nada.
5. ¿Dónde deben vivir los prompts de un flujo de trabajo?
    - Como archivos guardados, con versión y revisados, en un lugar compartido y aprobado; no escritos a mano en cada flujo.

6. Un reporte en las entradas dice "ignora tus reglas". ¿Qué dos partes del flujo impiden que eso se entregue?
    - La revisión de validación, que marca enlaces, destinatarios o acciones inesperados, y el punto de aprobación humana. El caso difícil del conjunto de prueba también lo detecta antes de la liberación.

Sesión 7 · Herramientas, MCP e inyección de prompts

Video: 9:12 (enlace por agregar al publicar) · Para quienes automatizan con IA · YouTube: "MCP e inyección de prompts: herramientas de IA seguras | Capacitación en IA #7"

Puntos clave

- El modelo solo pide una herramienta; el arnés decide y la ejecuta. Las lecturas de fuentes aprobadas corren solas, las acciones esperan a una persona, y todo resultado regresa como dato, no como instrucción.
- MCP (Model Context Protocol) es el estándar abierto para conectar herramientas a cualquier arnés. Construye un servidor y una app de chat, un flujo de trabajo y un agente lo pueden usar.
- La inyección de prompts, instrucciones escondidas en contenido que la IA lee, es el riesgo número uno para aplicaciones de IA según OWASP. Un ataque necesita datos privados, contenido no confiable y una salida; quita cualquiera de los tres y falla.
- Ninguna defensa sola la detiene; las capas sí. Las herramientas son software: la misma revisión, pruebas, escaneo y pipeline de liberación que cualquier código.

Capítulos: 0:00 Bienvenida: el modelo pide, el arnés decide · 0:30 Lo que vas a poder hacer · 0:55 Por qué las herramientas suben lo que está en juego · 1:23 La llamada a herramientas · 2:01 Las partes de un sistema de IA · 2:29 MCP: un servidor, muchos arneses · 3:05 Cómo funciona una llamada MCP · 3:41 Tres reglas para crear una herramienta · 4:02 La inyección de prompts · 5:03 Los tres ingredientes de un ataque · 5:33 Defensas en capas · 6:20 Las herramientas son software · 6:49 La regla de oro · 7:07 Ejercicio: síguela y ponle un punto de control · 8:03 Pon a prueba lo aprendido · 8:46 Lo que sigue: Agentes, skills y cómo dirigir el programa (Sesión 8)

Pon a prueba lo aprendido

1. ¿Quién ejecuta una herramienta: el modelo o el arnés?
    - El arnés, es decir, tu código. El modelo solo pide.
2. ¿Por qué un servidor MCP puede atender una app de chat, un flujo y un agente?
    - MCP es un estándar abierto: cualquier arnés que lo hable puede usar el mismo servidor y sus herramientas, recursos y plantillas de prompt.
3. ¿Por qué hay que revisar las descripciones de las herramientas?
    - Los nombres y las descripciones son instrucciones que lee el modelo; una descuidada o maliciosa cambia su comportamiento.
4. ¿Qué es la inyección indirecta de prompts?
    - Instrucciones sembradas en un contenido que la IA lee después: un documento, una página web, un correo o un registro.
5. ¿Qué herramientas necesitan la aprobación de una persona?
    - Cualquier herramienta que envía, cambia o borra algo.

6. Nombra tres defensas contra la inyección de prompts.
    - Tres de estas: tratar el contenido como dato, acceso mínimo, aprobación para las acciones, validación de la salida, un contenedor limitado a endpoints aprobados, revisión línea por línea de las skills externas.

Sesión 8 · Agentes, skills y cómo dirigir el programa

Video: 8:17 (enlace por agregar al publicar) · Para quienes automatizan con IA y quienes dirigen el programa · YouTube: "Agentes de IA y skills: cómo dirigir el programa | Capacitación en IA #8"

Puntos clave

- Una skill es como una guía de inducción que la IA carga solo cuando la necesita. Al inicio solo se cargan los nombres y las descripciones, así que puedes instalar docenas sin gastar mucho. Una descripción vaga significa una skill que nunca se activa.
- Trata cada skill como código: léela, escanéala, fija la versión, instálala desde un registro interno y decide quién puede instalar.
- Una skill aporta el método, las herramientas aportan datos y acciones, y el modelo llena los huecos. Un agente corre en un contenedor, registra cada paso y espera en el punto de control hasta que una persona aprueba todo lo que sale.
- Todos empiezan en la Etapa 1, y cada etapa agrega un riesgo nuevo. La capacitación funcionó cuando el trabajo cambió: toma líneas base primero y luego mide el tiempo ahorrado, los errores detectados, el reúso y los flujos en producción.

Capítulos: 0:00 Bienvenida: juntamos las piezas · 0:26 Lo que vas a poder hacer · 0:50 Tres formas en que se estancan los programas de IA · 1:25 Cómo se carga una skill por capas · 1:59 Trata cada skill como código · 2:27 Skill + herramientas + modelo: un trabajo repetible · 3:14 Agentes con barreras · 3:51 Una biblioteca compartida de prompts y skills · 4:19 Cinco etapas, un riesgo nuevo en cada una · 5:11 Cómo medir lo que cambió · 5:52 La regla de oro · 6:05 Ejercicio: empaquétalo y planea el despliegue · 6:53 Pon a prueba lo aprendido · 7:32 Serie completa: tu siguiente paso

Pon a prueba lo aprendido

1. ¿Qué se carga antes de que se active una skill?
    - Solo el nombre y la descripción de cada skill, unos 80 tokens cada una.
2. ¿Qué etapa completan todos y dónde paran los analistas?
    - Todos completan la Etapa 1. Los analistas paran en la Etapa 3.
3. ¿Qué aporta cada uno: una skill, las herramientas y un modelo?
    - La skill aporta el método, las herramientas aportan datos, acciones y revisiones, y el modelo aporta criterio en los huecos.
4. ¿Dónde tiene que aprobar una persona el trabajo de un agente?
    - Antes de que cualquier acción salga del sistema: enviar, guardar de regreso o cambiar un registro.
5. ¿Qué hace que un prompt esté listo para la biblioteca?
    - Un archivo de texto plano con encabezado, la revisión de otra persona, una versión y casos de prueba que pasa.

6. Nombra dos medidas que muestren que la capacitación funcionó.
    - Dos de estas: tiempo ahorrado por tarea, errores detectados en revisión, reúso de la biblioteca, flujos en producción, hallazgos de seguridad.

Inmersiones técnicas

Dos inmersiones técnicas prácticas, con mentalidad DevSecOps, para quienes construyen y revisan. Van después de la Sesión 7 y se complementan: A construye el servidor; B lo empaca, lo escanea, lo firma y lo publica.

Inmersión técnica A · Construye un servidor MCP desde cero

Video: 25:21 (enlace por agregar al publicar) · Para quienes construyen · YouTube: "Crea un servidor MCP desde cero (Python y TypeScript) | Inmersión técnica A"

Puntos clave

- La especificación de MCP cambió el 28 de julio de 2026: ya no hay saludo inicial ni sesiones, y cada solicitud lleva su propia versión del protocolo. Los dos SDKs oficiales pasaron a la versión 2 con APIs renombradas, así que los tutoriales anteriores son legado.
- Tres roles (host, cliente, servidor) y cuatro capas con fronteras de confianza. Cada capa aplica sus propias reglas, y los resultados siempre regresan como datos.
- Las herramientas, los recursos y los prompts los controlan el modelo, la app y el usuario. En la definición de una herramienta, el modelo lee el nombre y la descripción, el SDK aplica el esquema, y las anotaciones son solo indicaciones.
- Las acciones le preguntan primero a una persona con una solicitud de varias vueltas. Siempre maneja el "no" y el "cancelar", o el servidor se queda en un ciclo.
- stdio para una persona, Streamable HTTP detrás de un gateway para equipos, y contenedores con una lista de permitidos para el tráfico de salida en producción. El inicio de sesión remoto es OAuth 2.1; nunca reenvíes el token del usuario.
- Prueba como quien revisa (Inspector, la ruta de rechazo, un punto de control en CI con el snapshot de la lista de herramientas) y trátalo como un ciclo DevSecOps: diseñar, construir, probar, asegurar, publicar y operar.

Capítulos: 0:00 Bienvenida: un servidor en el que puedas confiar · 0:44 Lo que vas a poder hacer · 1:28 Por qué construirlo con cuidado · 2:33 Host, cliente y servidor · 3:34 El panorama completo: cuatro capas, tres fronteras de confianza · 4:32 Herramientas, recursos y prompts · 5:28 Anatomía de la definición de una herramienta · 6:17 Cómo se ve hoy una solicitud en el protocolo · 7:23 Preparación con los kits actuales · 8:31 Python: una herramienta de solo lectura en 16 líneas · 10:08 Python: una acción que primero le pregunta a una persona · 11:18 Cómo pregunta el servidor: solicitudes de varias vueltas · 12:28 TypeScript: la corrección del rechazo · 13:35 stdio vs. Streamable HTTP · 15:01 Dónde corre un servidor MCP · 15:58 Constrúyelo, inspecciónalo, rómpelo y corrígelo · 17:03 Conéctalo a un host · 18:04 OAuth 2.1, y por qué no hay token passthrough · 19:35 Amenazas y controles en el servidor · 21:07 DevSecOps para un servidor MCP · 22:15 La regla de oro · 22:33 Laboratorio práctico · 24:03 Pon a prueba lo aprendido · 24:59 Lo que sigue: contenedores, escaneo y el pipeline de CI/CD (Inmersión técnica B)

Pon a prueba lo aprendido

1. Sin saludo inicial, ¿cómo sabe un servidor la versión del protocolo del cliente?
    - Por los metadatos de cada solicitud, que llevan la versión del protocolo y las capacidades del cliente. Los clientes también pueden llamar primero a server/discover.

2. Tu servidor stdio funciona a mano, pero el host dice que falló. ¿Por qué?
   - Algo escribió en la salida estándar (un print o un console.log) y dañó el flujo del protocolo. Manda los logs a la salida de error, y revisa las rutas absolutas y las variables de entorno en la configuración del host.
3. destructiveHint es false. ¿El host puede aprobar la herramienta en automático por eso?
   - No. Las anotaciones son indicaciones no confiables y no aplican nada. Lo que aplica es la autorización del servidor, los scopes y la confirmación.
4. ¿Por qué un servidor nunca debe reenviar el token que recibió?
   - Eso es token passthrough (reenviar el token), y la especificación lo prohíbe. El token es para el servidor MCP; reenviarlo se salta los controles de lo que está atrás, rompe la auditoría y convierte al servidor en un proxy de exfiltración. Usa la credencial propia del servidor.
5. ¿Cómo obtiene un servidor la confirmación de una persona a mitad de una llamada?
   - Regresa un resultado de tipo input-required con un formulario de confirmación. El cliente le pregunta a la persona y reintenta la llamada con la respuesta y un id de solicitud nuevo.
6. Menciona dos controles contra un servidor malicioso o modificado.
   - Dos de estos: fijar versiones y digests exactos, instalar de publicadores verificados o de un espejo interno, guardar y comparar el snapshot de la lista de herramientas en CI, restringir el tráfico de salida, tokens con mínimo privilegio.

Inmersión técnica B · DevSecOps, contenedores, escaneo y el pipeline de CI/CD

Video: 23:39 (enlace por agregar al publicar) · Para quienes construyen y revisan · YouTube: "DevSecOps: contenedores, escaneo y pipeline de CI/CD | Inmersión técnica B"

Puntos clave

- DevSecOps significa seguridad en cada etapa, no un solo punto de control (en inglés, gate) al final: lleva la seguridad al inicio y sigue vigilando hasta producción.
- Una imagen es una caja sellada; su digest es su identidad real, mientras que una etiqueta se puede mover. La misma imagen corre en una laptop, un servidor, contenedores serverless, Kubernetes o el edge; cambia quién parcha qué, pero tu imagen, tu código, tus llaves y tus reglas de red siguen siendo tuyos.
- Endurece la imagen (base mínima fijada por digest, build multietapa, sin root) y mantén los secretos fuera de las imágenes y los repos: inyéctalos en tiempo de ejecución desde un gestor de secretos.
- Usa varios escáneres y luego haz el triaje por riesgo real: primero lo que se explota activamente (KEV), luego lo que probablemente se explotará (EPSS), luego lo crítico y alto que tiene corrección.
- Firma y atestigua el digest, verifícalo en la admisión, y promueve el mismo digest de staging a producción. Fija las actions de CI a SHAs de commit completos, porque tu escáner también es una dependencia.
- Firewalls: denegar por defecto en ambas direcciones, capa por capa. Un puerto publicado con Docker se salta ufw, y las cargas de IA solo deben llegar a la API del LLM y a los backends aprobados, con una lista de permitidos para el tráfico de salida (en inglés, egress).

Capítulos: 0:00 Bienvenida: una caja, confiable en todos lados · 0:37 Lo que vas a poder hacer · 1:05 Tres problemas que resuelve un buen pipeline · 1:47 DevSecOps: seguridad en cada etapa · 2:42 Imagen,

contenedor, registry y digest · 3:34 Dónde corren los contenedores · 4:50 Endurecer la imagen · 5:50 Lee un Dockerfile endurecido · 7:08 Dónde viven los secretos · 8:18 Un escáner en cada estación · 9:20 Herramientas de escaneo por categoría · 10:45 Triaje: corrige primero lo que ya se explota · 11:59 Firmar, atestar y verificar · 12:59 Herramientas de firma, registry, admisión y runtime · 14:12 La línea de CI/CD y sus puntos de control · 15:15 Un pipeline de GitHub Actions, anotado · 16:24 Cuando el escáner es el ataque · 17:20 Firewalls: denegar por defecto, en ambas direcciones · 18:34 Capas de red y sus herramientas · 20:04 Puntos de control específicos de IA · 20:56 La regla de oro · 21:13 Ejercicio: encuentra los problemas y haz el triaje · 22:19 Pon a prueba lo aprendido · 23:18 Cierre: constrúyelo, empácalo, publícalo

Pon a prueba lo aprendido

1. ¿Por qué firmar y desplegar por digest y no por etiqueta?
   - Una etiqueta es un rótulo que se puede mover; un digest es un hash de los bytes exactos. Garantiza que lo que se escaneó y aprobó es lo que corre.
2. Se agregó una llave en un paso y se borró en el siguiente. ¿Es seguro publicar la imagen?
   - No. Cada paso es una capa permanente, así que la llave sigue ahí. Rótala ya, reconstruye con inyección en tiempo de ejecución o montajes de secretos de build, y agrega escaneo de secretos.
3. Un escaneo muestra 40 hallazgos HIGH. ¿Qué corriges primero?
   - Primero KEV, luego EPSS alto, luego críticos y altos con corrección, enfocándote en lo que de verdad corre. Documenta el resto con VEX. Muchas veces una imagen base mínima más nueva es la corrección más rápida.
4. Escaneo contra firma: ¿por qué necesitas ambos?
   - El escaneo revisa el contenido en busca de problemas conocidos; la firma demuestra la identidad y el origen, y que nada cambió. Sin firma, alguien puede cambiar la caja; sin escaneo, publicas fielmente algo vulnerable.
5. ufw niega lo entrante, pero un puerto publicado con docker run -p sigue accesible. ¿Por qué? Da dos correcciones.
   - Docker publica los puertos con sus propias reglas NAT, que se saltan ufw. Enlaza el puerto a localhost detrás de un proxy inverso, o pon reglas en la cadena DOCKER-USER. Trata el grupo de seguridad de la nube como el perímetro real y prueba desde afuera con nmap.
6. Secuestraron la GitHub Action de un escáner. ¿Qué dos controles limitan el daño?
   - Fijar las actions a SHAs de commit completos (y las imágenes de los escáneres por digest), y mantener CI con mínimo privilegio: credenciales OIDC de vida corta, permisos mínimos y rotación rápida de secretos.